

```

"""
iPhone Deal Scout – Fichier unique tout-en-un
Copiez ce fichier sur GitHub sous le nom: bot.py
"""

import json, logging, random, re, smtplib, time, requests
from datetime import datetime
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from urllib.parse import urlencode, urljoin
from urllib.robotparser import RobotFileParser
from bs4 import BeautifulSoup

# — CONFIGURATION – MODIFIEZ ICI —————

CONFIG = {
    "rules": [
        {
            "model": "iPhone 14 Pro",
            "max_price": 500,
            "min_storage": 128,
            "platforms": ["vinted", "leboncoin"],
            "exclude": ["cassé", "pièces", "hs", "bloqué"]
        },
        {
            "model": "iPhone 13",
            "max_price": 350,
            "min_storage": 64,
            "platforms": ["vinted", "leboncoin"],
            "exclude": ["cassé", "pièces", "hs"]
        },
        {
            "model": "iPhone 15 Pro",
            "max_price": 700,
            "min_storage": 128,
            "platforms": ["vinted", "leboncoin"],
            "exclude": ["cassé", "pièces"]
        }
    ],
    "alerts": {
        # — Email (Gmail) —————
        # 1. Activez la validation 2 étapes sur Google
        # 2. Créez un mot de passe d'application sur myaccount.google.com
        # 3. Collez-le dans smtp_password (pas votre vrai mot de passe !)
    }
}

```

```

    "email_enabled": False,
    "smtp_user": "votre.email@gmail.com",
    "smtp_password": "xxxx xxxx xxxx xxxx",    # mot de passe d'app Gmail
    "recipient": "destinataire@email.com",

    # — Pushover (notification mobile gratuite) —————
    # 1. Créez un compte sur pushover.net
    # 2. Installez l'app Pushover sur votre iPhone
    # 3. Copiez votre User Key et créez un App Token
    "pushover_enabled": False,
    "pushover_user_key": "VOTRE_USER_KEY",
    "pushover_api_token": "VOTRE_APP_TOKEN",
},
"interval_minutes": 15,    # fréquence de scan
"min_score": 60,          # score minimum pour alerter (60 = ~30% sous le pri
}

# — PRIX MARCHÉ DE RÉFÉRENCE (€) —————

MARKET_PRICES = {
    "iphone 16 pro max": 1299, "iphone 16 pro": 1099, "iphone 16": 969,
    "iphone 15 pro max": 1099, "iphone 15 pro": 999, "iphone 15": 829,
    "iphone 14 pro max": 899, "iphone 14 pro": 799, "iphone 14": 629,
    "iphone 13 pro":      649, "iphone 13":      499, "iphone 12 pro": 449,
    "iphone 12":          349, "iphone se":      259,
}

# — LOGGING —————

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.StreamHandler(), logging.FileHandler("bot.log", encoding="u
)
log = logging.getLogger("DealBot")

# — HELPERS —————

USER_AGENTS = [
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Chrome/124.0.0.
    "Mozilla/5.0 (Macintosh; Intel Mac OS X 14_4) AppleWebKit/605.1.15 Safari/605
    "Mozilla/5.0 (X11; Linux x86_64; rv:125.0) Gecko/20100101 Firefox/125.0",
]

_robots_cache = {}

def can_fetch(base_url, path):

```

```

    if base_url not in _robots_cache:
        rp = RobotFileParser()
        rp.set_url(urljoin(base_url, "/robots.txt"))
        try: rp.read()
        except: return True
        _robots_cache[base_url] = rp
    return _robots_cache[base_url].can_fetch("*", path)

def make_session():
    s = requests.Session()
    s.headers.update({
        "User-Agent": random.choice(USER_AGENTS),
        "Accept-Language": "fr-FR,fr;q=0.9",
        "Accept": "text/html,application/xhtml+xml,*/*;q=0.8",
    })
    return s

def polite_get(session, url, **kwargs):
    time.sleep(random.uniform(3, 10))
    try:
        r = session.get(url, timeout=15, **kwargs)
        r.raise_for_status()
        return r
    except Exception as e:
        log.warning(f"Erreur requête {url}: {e}")
        return None

def get_market_price(title):
    t = title.lower()
    for model, price in sorted(MARKET_PRICES.items(), key=lambda x: -len(x[0])):
        if model in t:
            return price
    return None

def score_deal(price, title):
    market = get_market_price(title)
    if not market: return 50, 0, None
    disc = (1 - price / market) * 100
    score = min(100, max(0, round(50 + disc * 0.833)))
    return score, round(disc, 1), market

# — SCRAPERS —————

def scrape_vinted(session, rule):
    base = "https://www.vinted.fr"
    if not can_fetch(base, "/api/v2/catalog/items"):
        log.warning("Vinted: bloqué par robots.txt")

```

```

        return []
    params = {
        "search_text": rule["model"], "price_to": rule["max_price"],
        "catalog_ids": "2,16", "order": "newest_first", "per_page": 40,
    }
    url = f"{base}/api/v2/catalog/items?urlencode(params)"
    r = polite_get(session, url, headers={"Accept": "application/json"})
    if not r: return []
    try:
        items = r.json().get("items", [])
    except:
        return []
    results = []
    for item in items:
        price_raw = item.get("price", {})
        price = float(price_raw.get("amount", 0)) if isinstance(price_raw, dict)
        if price <= 0 or price > rule["max_price"]: continue
        title = item.get("title", "")
        if rule["model"].lower() not in title.lower(): continue
        if any(kw.lower() in title.lower() for kw in rule.get("exclude", [])): co
        if rule.get("min_storage"):
            m = re.search(r"(\d+)\s*go", title.lower())
            if m and int(m.group(1)) < rule["min_storage"]: continue
        score, disc, market = score_deal(price, title)
        results.append({
            "platform": "Vinted", "title": title, "price": price,
            "url": f"{base}/items/{item.get('id')}",
            "location": item.get("city", ""),
            "score": score, "discount": disc, "market": market,
            "id": f"vinted_{item.get('id')}",
            "date": datetime.now().strftime("%H:%M"),
        })
    log.info(f"Vinted: {len(results)} résultat(s) pour '{rule['model']}'")
    return results

def scrape_leboncoin(session, rule):
    base = "https://www.leboncoin.fr"
    if not can_fetch(base, "/recherche"):
        log.warning("LeBonCoin: bloqué par robots.txt")
        return []
    params = {
        "text": rule["model"], "category": "43",
        "price": f"0-{int(rule['max_price'])}", "sort": "time", "order": "desc",
    }
    url = f"{base}/recherche?urlencode(params)"
    r = polite_get(session, url)
    if not r: return []

```

```

soup = BeautifulSoup(r.text, "html.parser")
script = soup.find("script", {"id": "__NEXT_DATA__"})
if not script: return []
try:
    ads = json.loads(script.string).get("props", {}).get("pageProps", {}).get
except:
    return []
results = []
for ad in ads:
    price_raw = ad.get("price", [None])[0] if ad.get("price") else None
    price = float(price_raw) if price_raw else 0
    if price <= 0 or price > rule["max_price"]: continue
    title = ad.get("subject", "")
    if rule["model"].lower() not in title.lower(): continue
    if any(kw.lower() in title.lower() for kw in rule.get("exclude", [])): co
    if rule.get("min_storage"):
        m = re.search(r"(\d+)\s*go", title.lower())
        if m and int(m.group(1)) < rule["min_storage"]: continue
    score, disc, market = score_deal(price, title)
    results.append({
        "platform": "LeBonCoin", "title": title, "price": price,
        "url": urljoin(base, ad.get("url", "")),
        "location": ad.get("location", {}).get("city", ""),
        "score": score, "discount": disc, "market": market,
        "id": f"lbc_{ad.get('list_id')}",
        "date": datetime.now().strftime("%H:%M"),
    })
log.info(f"LeBonCoin: {len(results)} résultat(s) pour '{rule['model']}'")
return results

```

— ALERTES —

```

def send_email(cfg, deals):
    if not cfg.get("email_enabled") or not deals: return
    rows = ""
    for d in deals:
        rows += f"""
<tr>
    <td style="padding:12px;border-bottom:1px solid #1e293b;">
        <a href="{d['url']}" style="color:#38bdf8;font-weight:bold;">{d['titl
        <small style="color:#94a3b8;">{d['platform']} · {d['location']} · {d[
    </td>
    <td style="padding:12px;border-bottom:1px solid #1e293b;text-align:right
        <strong style="color:#f1f5f9;font-size:1.1em;">{d['price']:.0f} €</st
        <span style="color:#22c55e;font-weight:bold;">-{d['discount']}%</span
    </td>
    <td style="padding:12px;border-bottom:1px solid #1e293b;text-align:cent

```

```

        <span style="background:#0f172a;color:#38bdf8;padding:4px 10px;border
            {d['score']}/100
        </span>
    </td>
</tr>""
html = f"""<html><body style="background:#0f172a;font-family:sans-serif;color
<div style="max-width:680px;margin:32px auto;background:#1e293b;border-radius
    <div style="background:linear-gradient(135deg,#0ea5e9,#6366f1);padding:24px
        <h1 style="margin:0;font-size:1.3em;">📱 iPhone Deal Scout - {len(deals)}
    </div>
<table style="width:100%;border-collapse:collapse;">
    <thead><tr style="background:#0f172a;">
        <th style="padding:10px 12px;text-align:left;color:#64748b;font-size:0.
        <th style="padding:10px 12px;text-align:right;color:#64748b;font-size:0
        <th style="padding:10px 12px;text-align:center;color:#64748b;font-size:
    </tr></thead>
    <tbody>{rows}</tbody>
</table>
<p style="padding:16px 28px;color:#475569;font-size:0.8em;">
    {datetime.now().strftime("%d/%m/%Y à %H:%M")} · iPhone Deal Scout Bot
</p>
</div></body></html>"""
try:
    msg = MIMEMultipart("alternative")
    msg["Subject"] = f"🔥 {len(deals)} bonne(s) affaire(s) iPhone!"
    msg["From"] = cfg["smtp_user"]
    msg["To"] = cfg["recipient"]
    msg.attach(MIMEText(html, "html", "utf-8"))
    with smtplib.SMTP("smtp.gmail.com", 587) as s:
        s.starttls()
        s.login(cfg["smtp_user"], cfg["smtp_password"])
        s.sendmail(cfg["smtp_user"], cfg["recipient"], msg.as_string())
    log.info(f"Email envoyé à {cfg['recipient']}")
except Exception as e:
    log.error(f"Erreur email: {e}")

def send_pushover(cfg, deals):
    if not cfg.get("pushover_enabled") or not deals: return
    d = deals[0]
    try:
        requests.post("https://api.pushover.net/1/messages.json", data={
            "token": cfg["pushover_api_token"],
            "user": cfg["pushover_user_key"],
            "title": f"📱 Deal iPhone - {len(deals)} affaire(s)",
            "message": f"{d['title']}\n💰 {d['price']:.0f}€ (-{d['discount']}%) \n
            "url": d["url"], "url_title": "Voir l'annonce",
        }, timeout=10)

```

```

        log.info("Notification Pushover envoyée")
    except Exception as e:
        log.error(f"Erreur Pushover: {e}")

# ——— BOUCLE PRINCIPALE ———

def main():
    rules = CONFIG["rules"]
    alerts = CONFIG["alerts"]
    interval = CONFIG["interval_minutes"] * 60
    min_score = CONFIG["min_score"]
    alerted = set()
    session = make_session()

    log.info(f"🚀 Bot démarré - {len(rules)} règle(s) - scan toutes les {CONFIG['interval_minutes']} min...")

    while True:
        new_deals = []
        for rule in rules:
            log.info(f"🔍 Scan: {rule['model']} (max {rule['max_price']}€)")
            listings = []
            if "vinted" in rule.get("platforms", []):
                listings += scrape_vinted(session, rule)
            if "leboncoin" in rule.get("platforms", []):
                listings += scrape_leboncoin(session, rule)

            for deal in listings:
                if deal["id"] in alerted: continue
                if deal["score"] < min_score: continue
                new_deals.append(deal)
                alerted.add(deal["id"])

        if new_deals:
            new_deals.sort(key=lambda x: -x["score"])
            log.info(f"✅ {len(new_deals)} nouvelle(s) affaire(s) trouvée(s)!")
            for d in new_deals:
                log.info(f" [{d['platform']}] {d['title']} - {d['price']}€ (score {d['score']})")
                send_email(alerts, new_deals)
                send_pushover(alerts, new_deals)
        else:
            log.info("Aucune nouvelle affaire ce cycle.")

        log.info(f"🕒 Prochain scan dans {CONFIG['interval_minutes']} min...")
        time.sleep(interval)

if __name__ == "__main__":
    main()

```

